



西安电子科技大学
XIDIAN UNIVERSITY

计算机科学与技术学院
School of Computer Science and Technology

分布式计算

Distributed Computing

主讲人：李龙海

第五讲 MapReduce模型和分布式计算框架



主要内容

- MapReduce并行计算模型
- 分布式并行计算框架
- Hadoop MapReduce分布式计算平台
- Hadoop平台上MapReduce程序设计
- 利用MapReduce实现大数据分析：单词计数示例

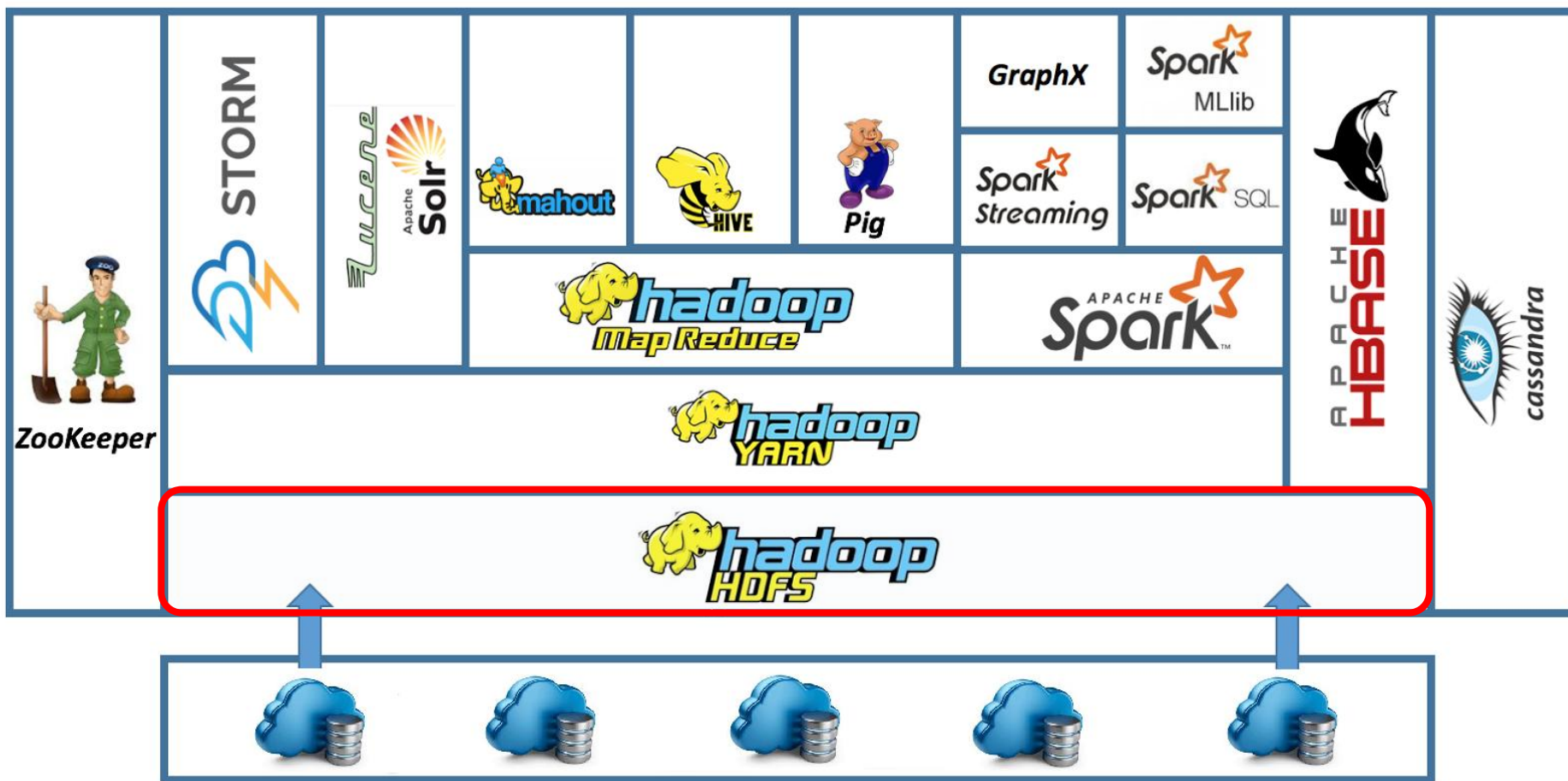


- 将面向大数据的计算任务划分多个子任务，将输入文件划分成数量相等的多个分片。每个子任务处理一个分片。
- 不同的子任务在不同的计算节点上运行。
- 最后将各个子任务的中间结果合并，计算出最终结果。





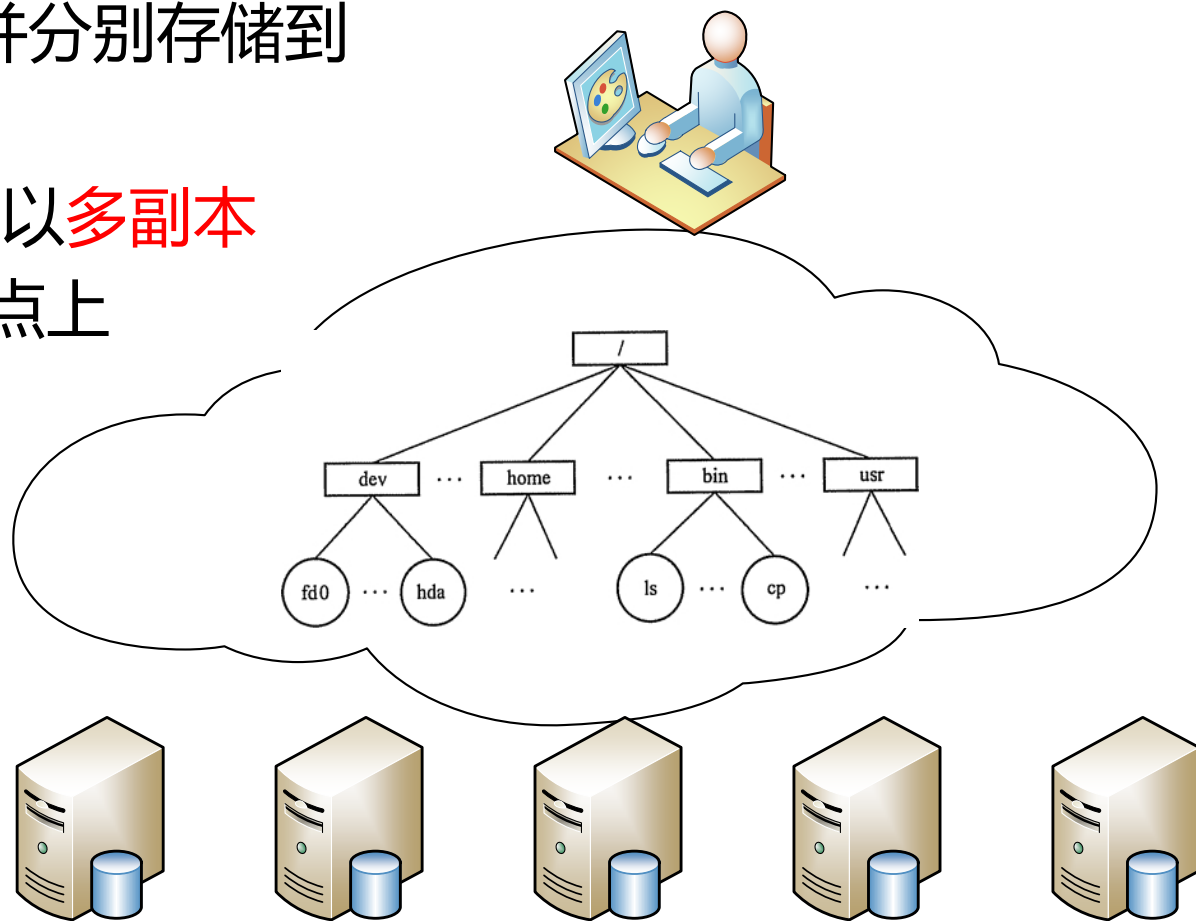
- **分布式计算平台**(或叫框架)和运行在平台之上的**应用层模块****相互协作**完成一个完整的分布式计算任务。
- **分布式计算平台的作用**:
 1. 将输入数据**自动分片**，将每个分片交给一个计算子任务处理；
 2. 将不同的计算子任务**分发**给不同的计算节点执行；
 3. 子任务分发时将考虑如下因素：各个计算节点的当前负载；尽量让计算子任务和对应的数据分片处于同一台物理主机。
 4. 帮助实现**中间计算结果的收集合并**、中间结果在不同计算节点之间的交换(混洗)；
 5. 容错性：监视各个子任务的执行状态，将执行失败的子任务重新调度给其它节点执行。
 6. 可扩展性：对集群中计算节点的增长或减少可以自适应
- **应用层模块的作用**：专注于实现框架要求下和具体应用层逻辑相关的子任务计算逻辑。



- Hadoop平台：面向大数据的**开源分布式平台**，包含了收集、存储、批处理、实时处理、分析和可视化大数据的一系列软件系统。



- 分布式文件系统：将分布式系统中多个节点的存储资源整合在一起，向用户/应用程序呈现**统一的存储空间和文件系统目录树**。用户无需关心数据存储在哪个节点上。
- 大文件被**自动分块**并分别存储到不同的节点上。
- 每个**分块 (Block)** 以**多副本**方式存放在不同节点上

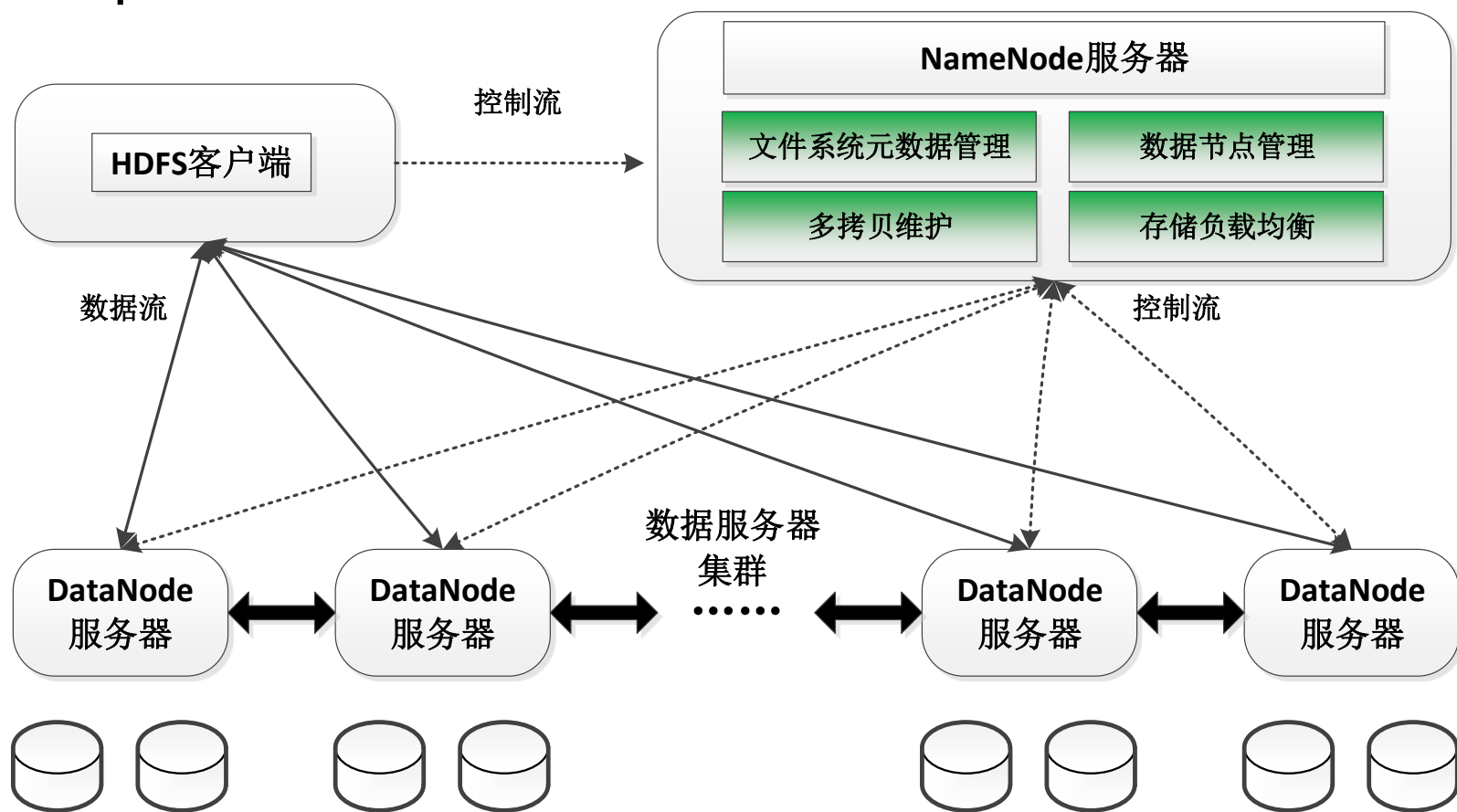




- HDFS 遵循主/从架构，由单个 NameNode(NN, 主节点) 和多个 DataNode(DN, 从节点) 组成。
- 高容错：采用数据多副本方案，部分硬件的损坏不会导致全部数据的丢失。
- 高吞吐量：支持多个客户端同时与多数据节点直接通信，实现了高吞吐量的数据访问。（读数据吞吐量高）
- 大文件支持：HDFS 适合于大文件的存储，文档的大小应该是 GB 到 TB 级别的。（为什么不适合大量小文件存储？）
- 简单一致性模型：HDFS 更适合于一次写入多次读取 (write-once-read-many) 的访问模型。支持将内容追加到文件末尾，不能从文件任意位置新增数据。

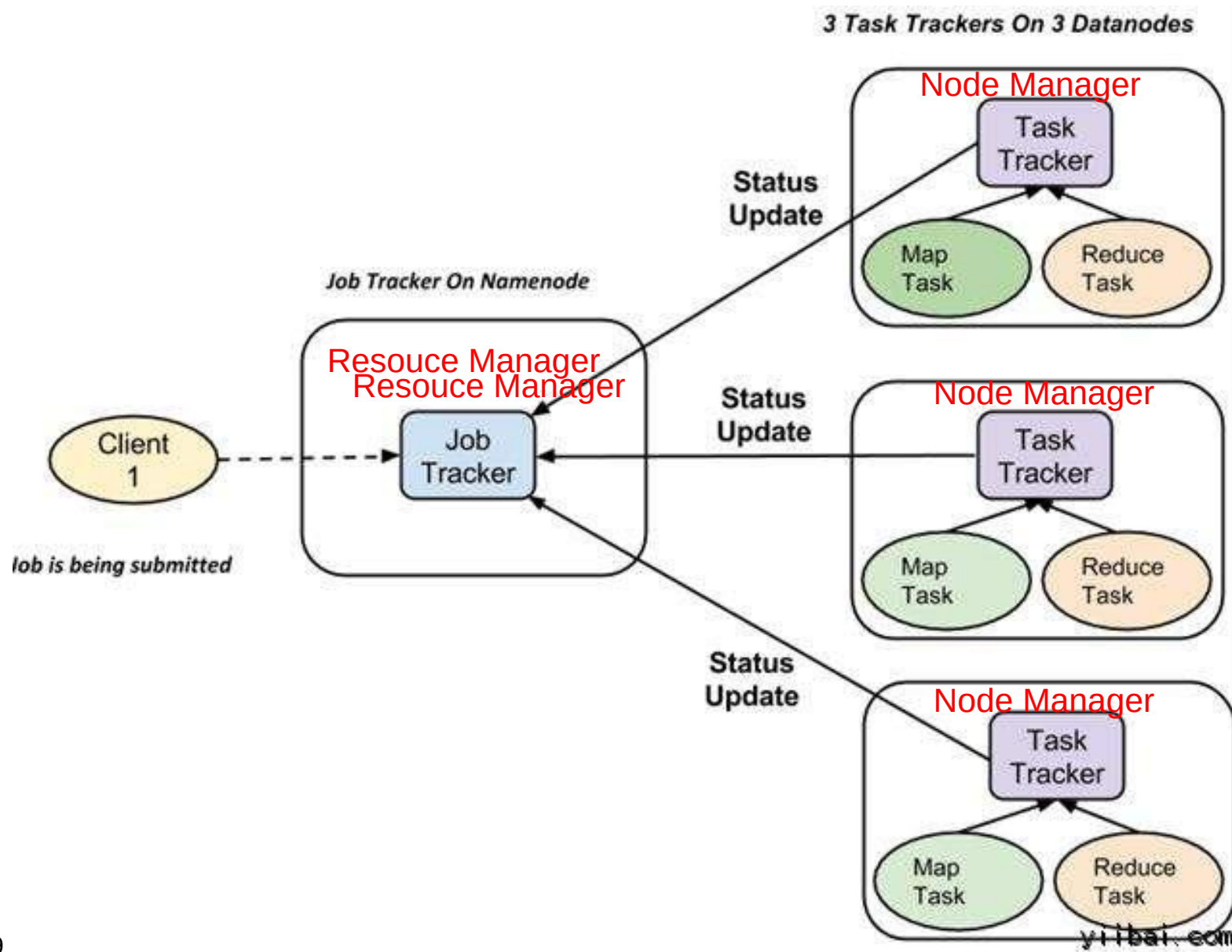


- **主节点**负责存储文件系统的**元数据**（目录信息、文件名、每个文件由哪些分块组成等）
- 从节点负责存储每个文件**实体数据**（文件的**分块**数据）
- 与MapReduce分布式计算框架紧密集合



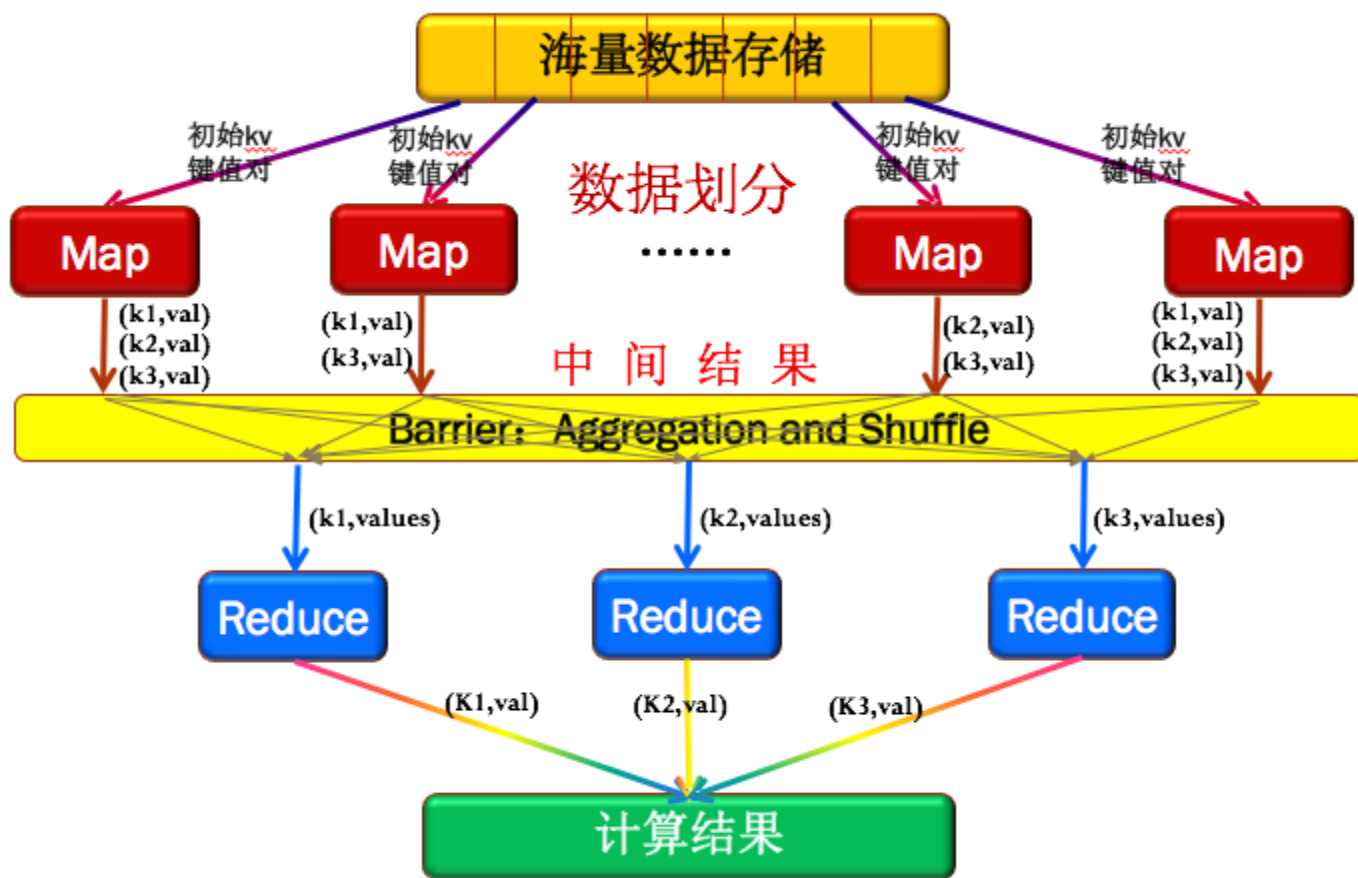


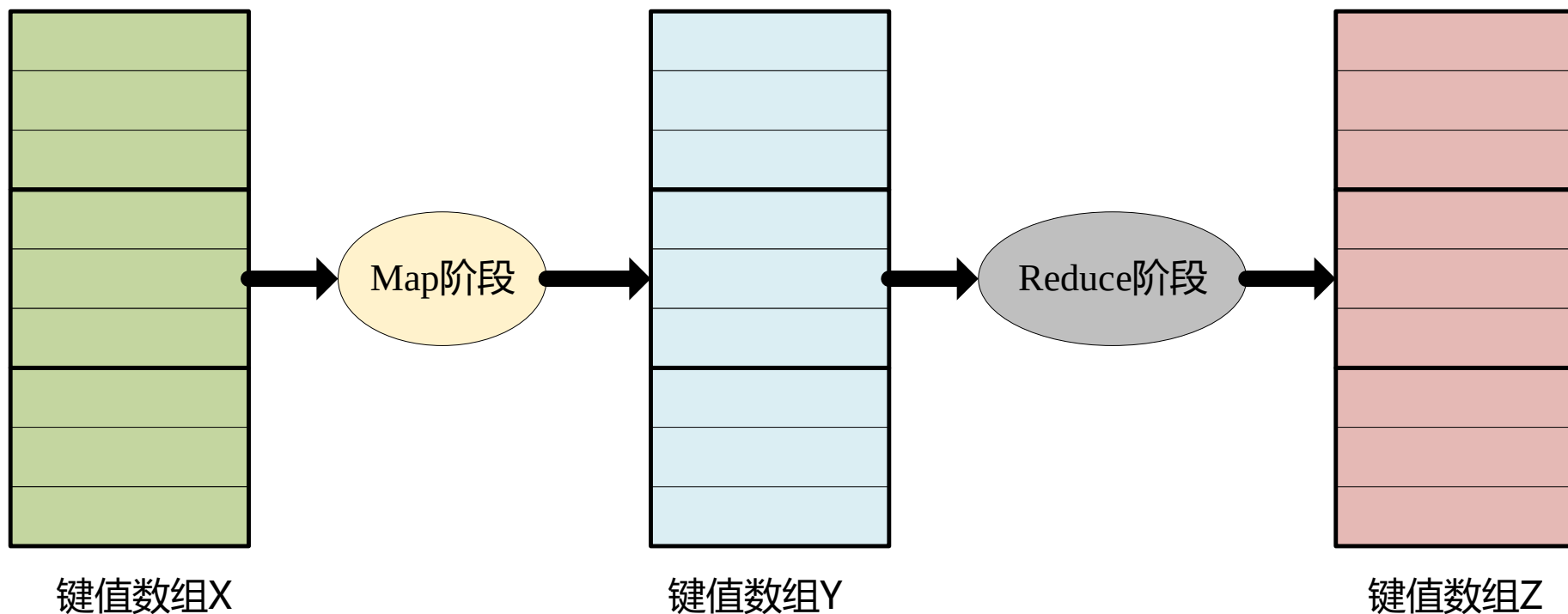
Hadoop MapReduce分布式平台





- 将大数据并行计算任务划分为三个阶段：Map阶段、聚集混洗阶段（含二次分区），Reduce阶段。





- 三个键值数组X、Y、Z的Key的类型和Value的类型不一定相同；
- 键值数组X经过Map转化成了键值数组Y，X和Y之间可以是一对一、一对多和多对一的关系。
- 键值数组Y再经过Reduce转化成了键值数组Z，Y和Z之间可以是一对一、一对多和多对一的关系。



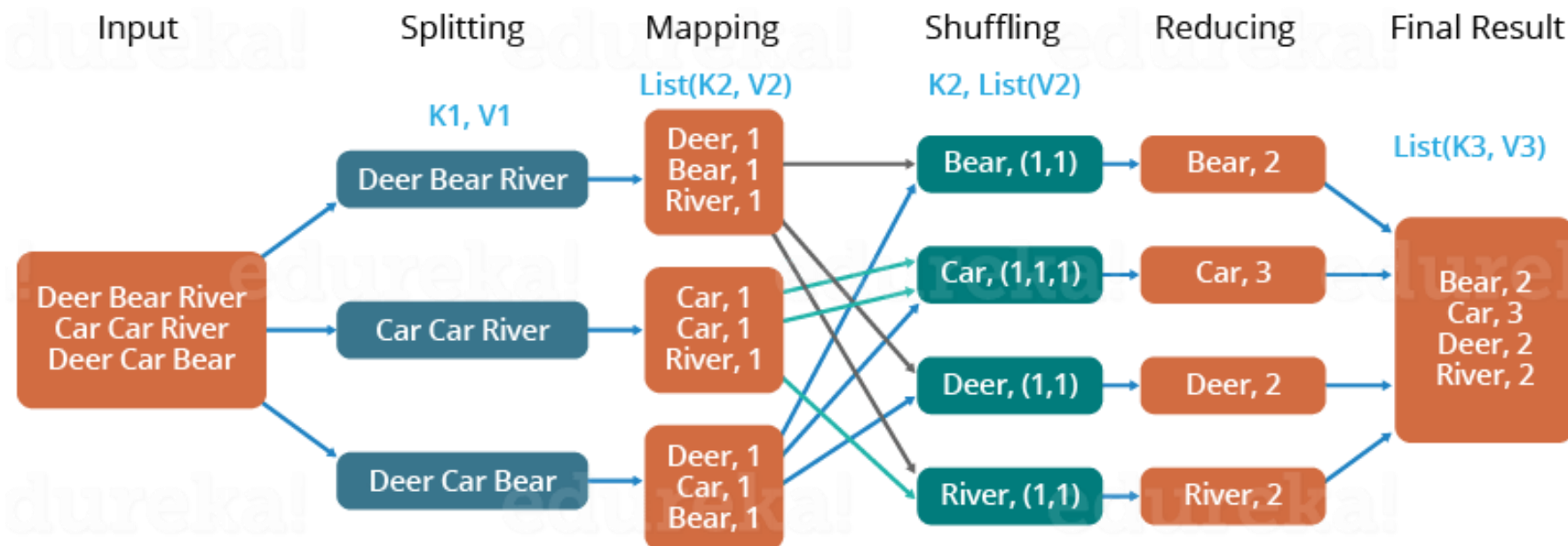
- **Map阶段**：第一阶段并行，将输入文件（或目录下的多个输入文件）划分成多个分区，每个分区都交给一个独立的Map子任务进行处理。
- Map子任务的从输入分片中按配置规则逐个提取“行记录”，并规整为键值数组： $\{<\text{行id}, \text{行数据}>\}$ ，Map子任务输出一个键值数组 $\{<\text{key}, \text{value}>\}$ （输出中key、value的格式、类型和内容由Map程序自己决定）
- **聚集混洗阶段**：平台自动将不同Map子任务输出的 $<\text{key}, \text{value}>$ 数组按照key自动进行聚集，聚集成数组 $\mathbf{A} = \{<k1, [v1, v2, \dots]>, <k2, [v3, v4, \dots]>, <k3, [v5, v6, \dots]>, \dots\}$ 的形式。
- **Reduce阶段**：第二阶段并行，将聚集之后的数组A划分成多个分区，每个分区都交给一个独立Reduce子任务进行处理。（分区内按key进行排序）。Reduce子任务根据输入生成新的键值数组 $\mathbf{B} = \{<\text{new-key}, \text{new-value}>\}$ 。如果Reduce阶段输出不是最终结果，还可以以B数组为输出启动新一轮MapReduce过程。
- **要点**：通过Map子任务输出的key值部分控制不同子任务输出结果的相互交换和聚集。



■ 单词计数统计的例子

The Overall MapReduce Word Count Process

edureka!





下列问题如何用MapReduce模型实现？

- 在网页数据库中统计不同网页被引用的频率。
- 根据网页数据库生成关于每个网页的逆向链接统计。（<target, list(source)>）
- 生成关于网页数据库的全文检索索引（倒排索引）。
- 海量数据的全文查找或关键字匹配
- 数据过滤
- 数据去重
- 数值数组计算总和、平均值、最大最小值、TopN
- 计算数字图像直方图
- 海量数据的随机采样
- 给出child-parent数据表，要求输出grandchild-grandparent数据集表



Hadoop上一个MapReduce Job的详细执行过程：

- **平台将输入文件分片**：分片大小、方式**可定制**，默认按HDFS分块大小分片
- **平台针对每个分片创建一个Map子任务**：该Map子任务从**文件分片**中以“**行记录**”的形式逐条读取，Map子任务根据具体计算需求输出一个或多个**(k, v)对**。每读入一条记录，就调用一次Mapper对象的map方法。(文件分片中数据组织成**行记录**的方式可定制，默认遇到换行符\n为一行；默认将行记录规整为行号为Key，行内容为Value的形式)
- **二次分片**：Map Task 执行完毕后生成一个(key, value)数组。**平台**将Map Task中间计算结果**二次分片**：二次分片逻辑可定制，默认方式是将key值取Hash然后**模除**Reduce Task节点数。
- 框架将每个二次分片内的<key, value>数组进行分片内**排序**。

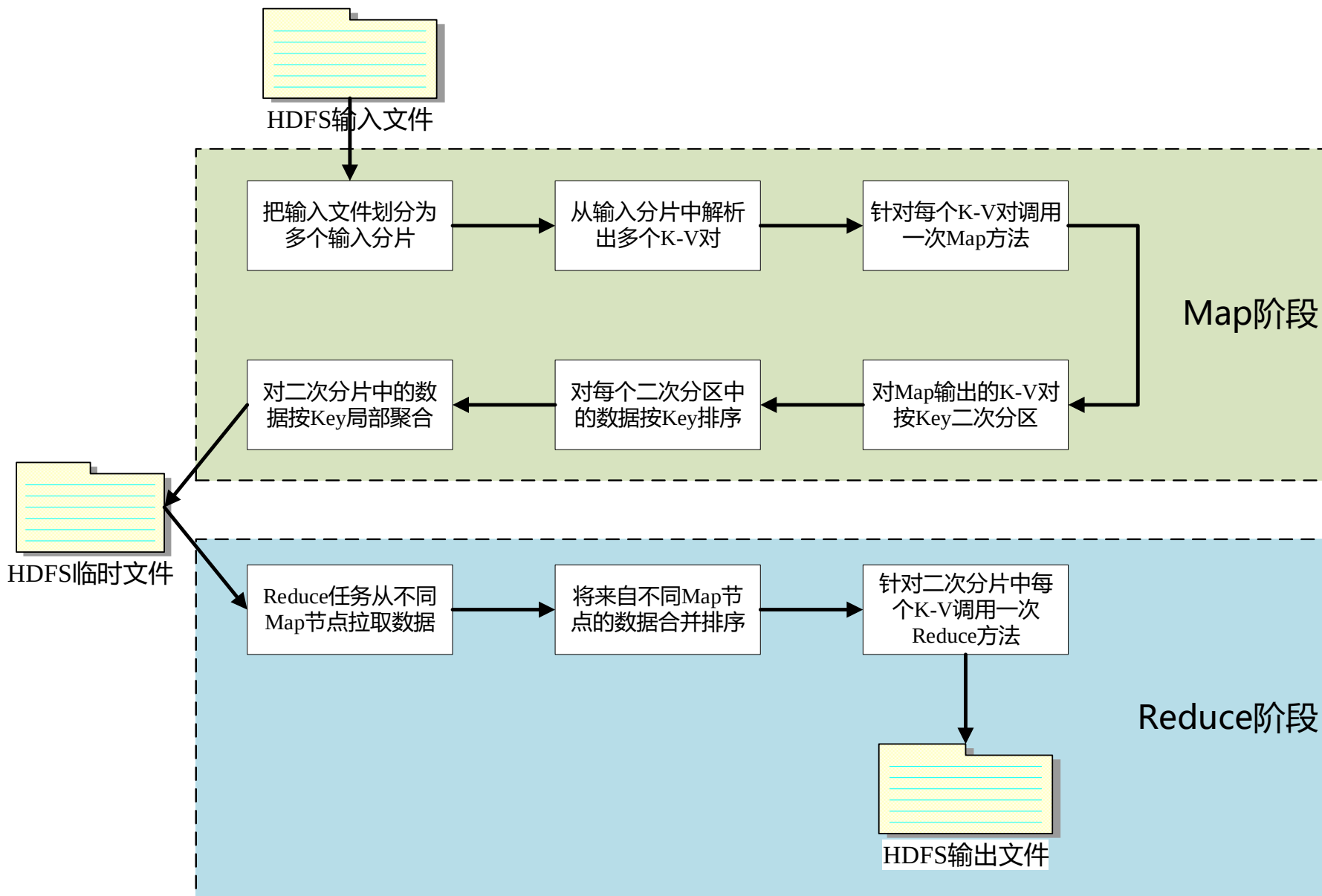


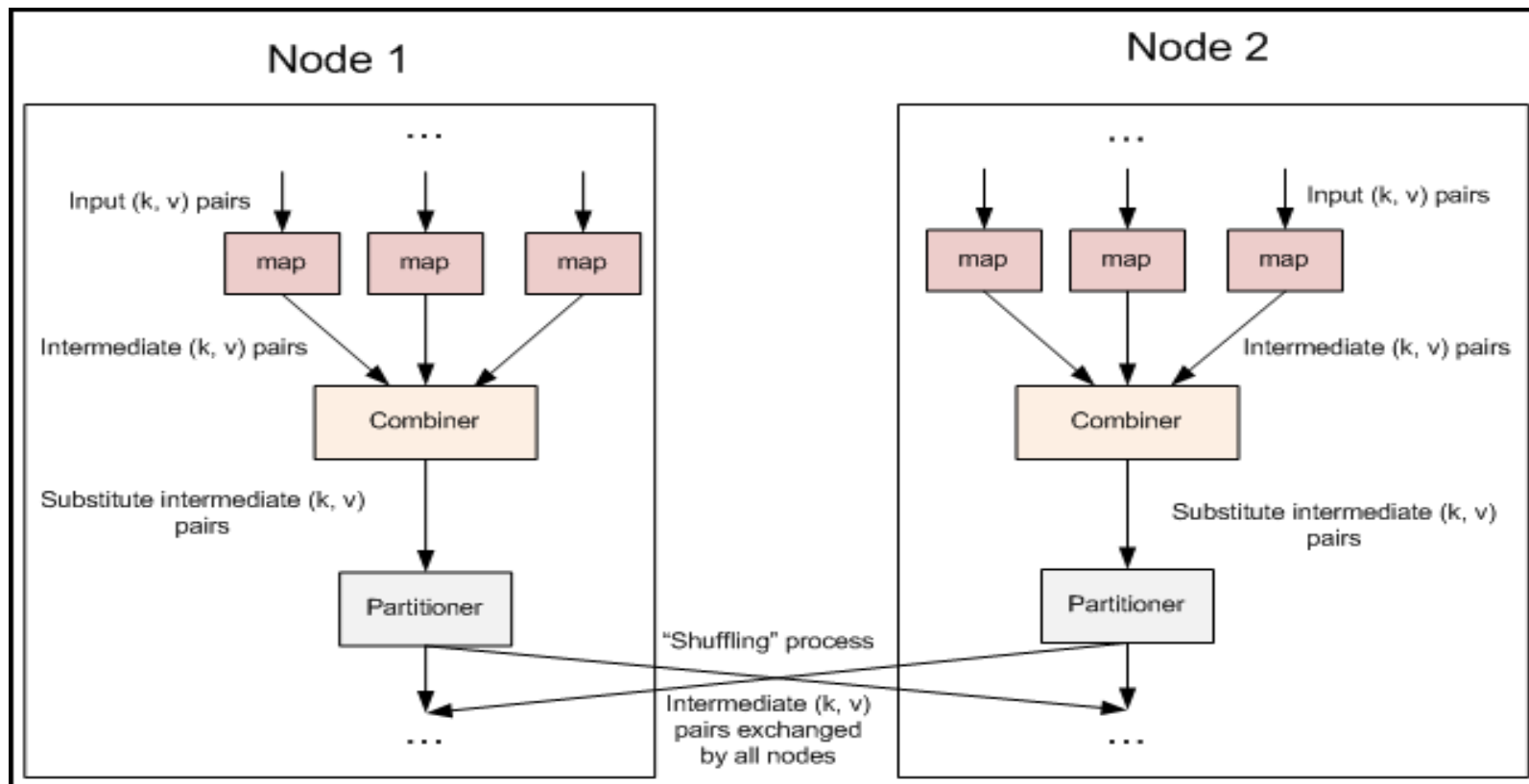
Hadoop上一个MapReduce Job的详细执行过程：

- **局部聚合**：框架将每个二次分片内的元素按key值进行局部聚合。局部聚合处理逻辑可通过Combiner对象定制。默认逻辑是将 $\langle k1, v1 \rangle$ 、 $\langle k1, v2 \rangle$ 、 $\langle k1, v3 \rangle$... 聚合成 $\langle k1, [v1, v2, v3] \rangle$ 。
- 经过排序和局部聚合之后的二次分片数据被**暂**存入HDFS。
- 框架针对用户对Reduce Task数目的设定创建n个Reduce Task。
(设定方法： `conf.setNumReduceTasks(int n)`)
- **Reduce阶段**：框架将每个二次分片分发给一个Reduce Task。该Reduce Task从二次分片每取出一条(key, $\langle v1, v2, v3 \dots \rangle$)形式的记录，就调用一次Reducer对象的reduce方法，该方法生成最终结果。



MapReduce详细工作过程





map: $(K1, V1) \rightarrow \text{list}(K2, V2)$

combine: $(K2, \text{list}(V2)) \rightarrow \text{list}(K3, V3)$

reduce: $(K3, \text{list}(V3)) \rightarrow \text{list}(K4, V4)$



Client类

1. 设置工作参数
2. 创建Map Reduce Job对象
3. 设置要上传给Hadoop平台的Jar包或Class
4. 设置Mapper类
5. 设置Combiner类
6. 设置Reducer类
7. 设置输出记录的形式
8. 设置输入文件路径、输出文件路径
9. 启动该Job并等待其执行完毕

Mapper类

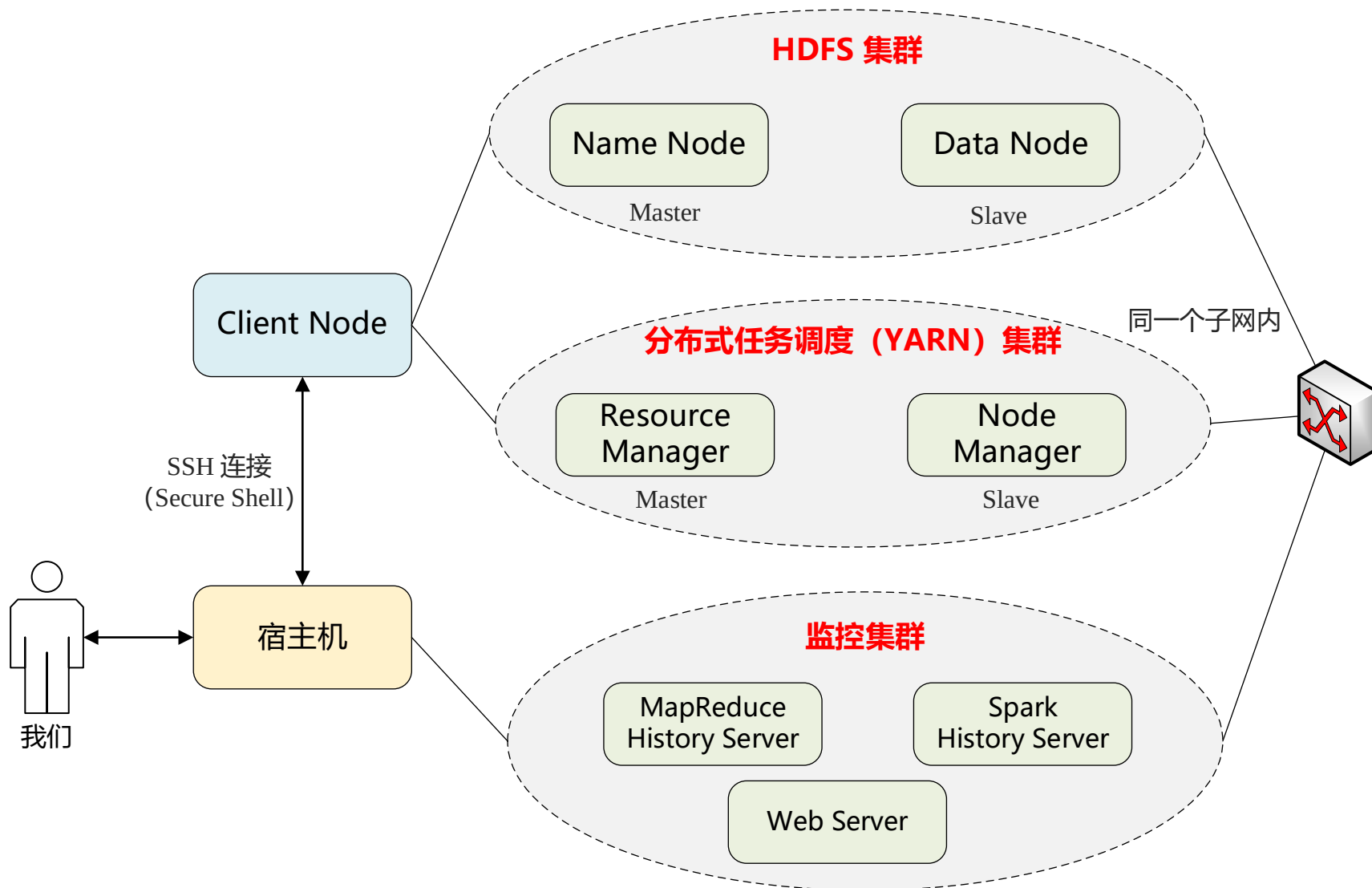
实现map方法: $(K1, V1) \rightarrow \text{list}(K2, V2)$

Combiner类

实现reduce方法: $(K2, \text{list}(V2)) \rightarrow \text{list}(K3, V3)$

Reducer类

实现reduce方法: $(K3, \text{list}(V3)) \rightarrow \text{list}(K4, V4)$



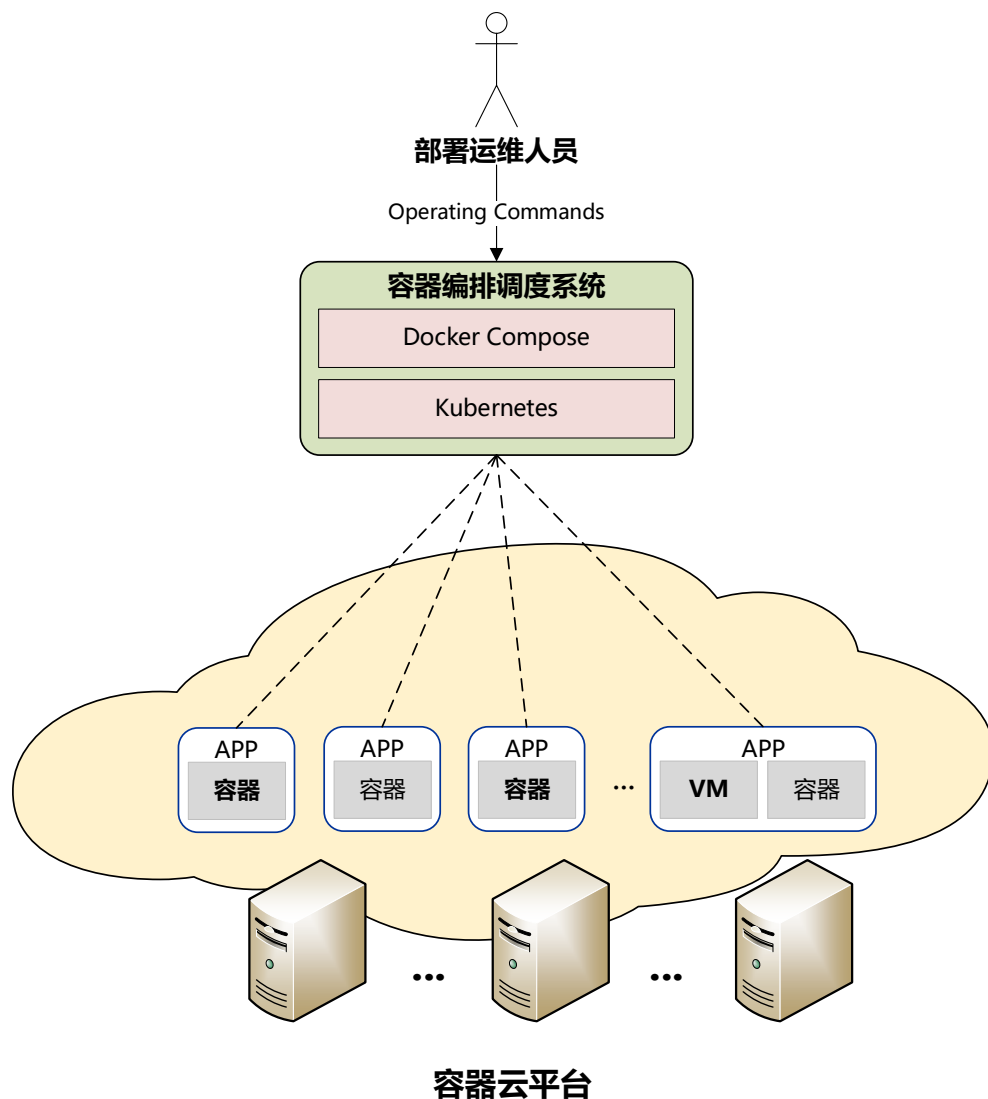


Docker容器与传统虚拟机的区别



容器为运行于其中的应用虚拟化出如下“独立”资源：

- (1) 文件系统
- (2) 网络环境 (IP地址、路由表、主机名、子网...)
- (3) 应用运行时环境 (依赖库、环境变量、配置文件...)
- (4) ...



分布式系统的OS:

- 分发容器镜像
- 按需启动、关停容器
- 按需迁移容器
- 资源调度（存储、网络）
- 负载均衡
- 自动扩容、缩容
- 自动故障恢复



■ 题目1:

输入文件为学生成绩信息，包含了必修课与选修课成绩，格式如下：

班级1, 姓名1, 科目1, 必修, 成绩1
 （注：
 为换行符）

班级2, 姓名2, 科目1, 必修, 成绩2

班级1, 姓名1, 科目2, 选修, 成绩3

.....,,,

编写Hadoop平台上的MapReduce程序，实现如下功能：

1. 计算每个学生必修课的平均成绩。
2. 按科目统计每个班的平均成绩。



■ 题目2:

输入文件的每一行为具有父子/父女/母子/母女/关系的一对人名，例如：

Tim, Andy

Harry, Alice

Mark, Louis

Andy, Joseph

.....,

假定不会出现重名现象。

编写Hadoop平台上的MapReduce程序，找出所有具有grandchild-grandparent关系的人名组。

■ 提交:



1. 《The Google file system》, 2003
2. 《MapReduce: simplified data processing on large clusters》, 2008
3. 《The Hadoop Distributed File System》, 2010